

Logo
SUP'COM

Université de Carthage

Logo
organisme d'accueil

École Supérieure des Communications de Tunis (SUP'COM)

Cycle d'ingénieur : Génie des Réseaux et Systèmes Informatiques

RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du Diplôme National d'Ingénieur

DevOps Central Platform

Conception et mise en œuvre d'une chaîne DevSecOps
et GitOps de bout en bout sur cluster Kubernetes

Réalisé par

Abdelkader Ben Hmida

Encadrant académique

[Nom de l'encadrant académique]

Encadrant professionnel

[Nom de l'encadrant professionnel]

Membres du jury

Président : [Nom, grade]

Rapporteur : [Nom, grade]

Encadrant : [Nom de l'encadrant académique]

Année universitaire 2025 – 2026

Résumé

Ce projet de fin d'études porte sur la conception, la réalisation et la validation d'une plateforme DevOps complète, nommée *DevOps Central Platform*, couvrant l'intégralité du cycle de vie d'une application distribuée : du provisionnement de l'infrastructure jusqu'à la supervision en production.

La plateforme repose sur une infrastructure virtualisée libvirt/KVM décrite entièrement en Infrastructure as Code avec Terraform, configurée de manière idempotente par Ansible, et hébergeant un cluster Kubernetes 1.28 multi-nœuds installé par kubeadm avec le plugin réseau Calico. La charge applicative est constituée de trois microservices Python FastAPI conteneurisés selon un modèle multi-stage durci, déployés via Kustomize sur trois environnements distincts (dev, staging, production).

La sécurité est intégrée à chaque étape de la chaîne plutôt qu'ajoutée a posteriori : détection de secrets par Gitleaks, analyse de vulnérabilités des images par Trivy, audit des dépendances par pip-audit, politiques d'admission conftest et Kyverno, NetworkPolicies en *default-deny*, Pod Security Admission en mode *restricted*, et gestion dynamique des secrets par HashiCorp Vault selon un contrat *fail-closed*. L'automatisation du déploiement suit un modèle hybride : une chaîne d'intégration continue GitHub Actions en mode *push* pour la construction et la validation, et un moteur GitOps ArgoCD en mode *pull* pour la réconciliation continue entre l'état déclaré dans Git et l'état réel du cluster.

L'observabilité est assurée par une double chaîne : métriques (Prometheus, AlertManager, Grafana) et logs centralisés (Elasticsearch, Filebeat, Logstash, Kibana), avec des objectifs de niveau de service (SLO) formalisés en règles d'alerte exécutables. Sept scénarios de validation de bout en bout, incluant des scénarios d'incident volontairement provoqués, démontrent le comportement réel de la plateforme en condition nominale comme en condition dégradée.

Mots clés : DevOps, DevSecOps, GitOps, Kubernetes, Terraform, Ansible, Infrastructure as Code, conteneurisation, intégration continue, observabilité, SLO, gestion des secrets.

Table des matières

Résumé	i
Liste des acronymes	iv
Introduction générale	1
1 Contexte général et architecture	2
1.1 Présentation générale du projet	2
1.2 Méthodologie et périmètre de l'analyse	2
1.3 Vue d'ensemble de l'architecture	2
1.3.1 Les six dimensions de la plateforme	2
1.3.2 Flux de bout en bout	3
1.3.3 Inventaire rapide des technologies	3
1.4 Fiches d'identité des outils majeurs	4
1.5 Topologie réseau et dimensionnement	5
1.5.1 Réseau virtuel	5
2 Infrastructure as Code et configuration	7
2.1 Libvirt/KVM et cloud-init	7
2.2 Terraform — Infrastructure as Code	7
2.3 Ansible — configuration automatisée	8
3 Conteneurisation et orchestration	10
3.1 Docker — construction des images	10
3.2 containerd et Kubernetes 1.28 — socle du cluster	11
3.3 Manifests applicatifs — Kustomize	12
4 Architecture applicative et secrets	14
4.1 FastAPI + Uvicorn — le socle applicatif	14
4.2 Bibliothèque partagée shared/ et instrumentation	15
4.3 HashiCorp Vault — gestion dynamique des secrets	16
5 Intégration continue et DevSecOps	18
5.1 GitHub Actions — pipeline CI/CD	18
5.2 Gitleaks — détection de secrets	19
5.3 Trivy, pip-audit et pre-commit	20
5.4 Lint et validation des manifests	21
6 Déploiement continu GitOps	22

6.1	ArgoCD — déploiement déclaratif continu	22
7	Observabilité et SLO	24
7.1	Prometheus Operator, kube-state-metrics et kubelet	24
7.2	Carnet de requêtes PromQL et AlertManager	25
7.3	Grafana — visualisation dashboards-as-code	26
7.4	ELK Stack 8.14 — logs centralisés	27
7.5	Modèle de menaces simplifié et contre-mesures	28
7.6	SLO — objectifs de niveau de service	30
7.6.1	Le contrat chiffré	30
7.6.2	Implémentation bout en bout	30
7.6.3	Éléments de démonstration	31
8	Scénarios de validation	32
9	Exploitation et bilan	34
9.1	Scripts de validation de la plateforme	34
9.2	Tests de charge et index des runbooks	35
9.3	Tableau récapitulatif général	36
9.4	Limites connues et axes d'amélioration	38
	Conclusion générale et perspectives	39
	Bibliographie	41

Liste des acronymes

Acronyme	Signification
API	<i>Application Programming Interface</i>
CD	<i>Continuous Deployment</i> : déploiement continu
CI	<i>Continuous Integration</i> : intégration continue
CNI	<i>Container Network Interface</i>
CRD	<i>Custom Resource Definition</i>
CRI	<i>Container Runtime Interface</i>
CVE	<i>Common Vulnerabilities and Exposures</i>
DNS	<i>Domain Name System</i>
ELK	<i>Elasticsearch, Logstash, Kibana</i>
GHCR	<i>GitHub Container Registry</i>
HCL	<i>HashiCorp Configuration Language</i>
HPA	<i>Horizontal Pod Autoscaler</i>
IaC	<i>Infrastructure as Code</i>
JWT	<i>JSON Web Token</i>
KVM	<i>Kernel-based Virtual Machine</i>
KSM	<i>kube-state-metrics</i>
NAT	<i>Network Address Translation</i>
OCI	<i>Open Container Initiative</i>
OIDC	<i>OpenID Connect</i>
OOM	<i>Out Of Memory</i>
PDB	<i>Pod Disruption Budget</i>
PSA	<i>Pod Security Admission</i>
RBAC	<i>Role-Based Access Control</i>
SLA	<i>Service Level Agreement</i>
SLI	<i>Service Level Indicator</i>
SLO	<i>Service Level Objective</i>
SSH	<i>Secure Shell</i>
TLS	<i>Transport Layer Security</i>
VM	<i>Virtual Machine</i> : machine virtuelle
YAML	<i>YAML Ain't Markup Language</i>

Introduction générale

Contexte et problématique

Le mouvement DevOps désigne un ensemble de pratiques d'ingénierie visant à raccourcir le cycle entre une modification de code et sa mise à disposition fiable aux utilisateurs, en automatisant systématiquement ce qui peut l'être et en rendant observable ce qui ne peut pas l'être. Deux évolutions l'accompagnent : le **DevSecOps**, qui déplace les contrôles de sécurité vers l'amont de la chaîne, et le **GitOps**, qui fait du dépôt Git la source unique de vérité de l'état désiré.

La question à laquelle ce travail répond : *comment une équipe d'ingénierie fait-elle fonctionner une application en production de manière reproductible, fiable, sécurisée et observable, sans intervention manuelle ?*

Objectifs du projet

Construire une plateforme complète apportant une réponse concrète à chacune de ces questions, puis en démontrer le fonctionnement par des scénarios exécutables : décrire l'infrastructure en code versionné, automatiser la configuration de façon idempotente, conteneuriser une charge applicative selon les bonnes pratiques, intégrer une chaîne de contrôles de sécurité bloquante, déployer en GitOps avec réconciliation continue, instrumenter métriques et logs avec des SLO vérifiables, et valider l'ensemble par des scénarios de bout en bout incluant des défaillances provoquées.

Parti pris méthodologique : **documenter les échecs autant que les succès** : plusieurs décisions techniques présentées dans ce rapport proviennent d'incidents réellement rencontrés pendant la réalisation, chaque règle d'alerte portant une étiquette renvoyant au post-mortem correspondant. L'ensemble du contenu technique a été établi à partir de la lecture directe du code source effectivement actif dans le dépôt.

CHAPITRE 1

Contexte général, périmètre et architecture de la plateforme

1.1 Présentation générale du projet

DevOps Central Platform est une plateforme d'infrastructure DevOps de bout en bout construite autour de trois microservices **FastAPI** (*Users, Products, Orders*). L'application est un support volontairement simple : l'objectif réel du projet est de maîtriser la chaîne DevOps moderne telle que pratiquée en entreprise. Le projet tourne sur un **homelab libvirt/KVM**, sans service cloud facturé, tout en restant structurellement proche d'une production réelle.

Chaque brique du rapport apporte un élément de réponse à la question fondatrice : *reproductible* → Terraform, cloud-init, Ansible ; *fiable* → Kubernetes, probes, HPA, PDB ; *sécurisée* → Gitleaks, Trivy, pip-audit, Vault, NetworkPolicies, PSA restricted ; *observable* → Prometheus, AlertManager, Grafana, ELK ; *sans intervention manuelle* → GitHub Actions + ArgoCD.

1.2 Méthodologie et périmètre de l'analyse

Toutes les informations proviennent de la lecture directe du code source actif ; le dossier `docs/` a été **volontairement exclu** car ses spécifications sont obsolètes par rapport au code : le code est la seule source de vérité. Chaque figure de ce rapport correspond à une vérification exécutable sur la plateforme.

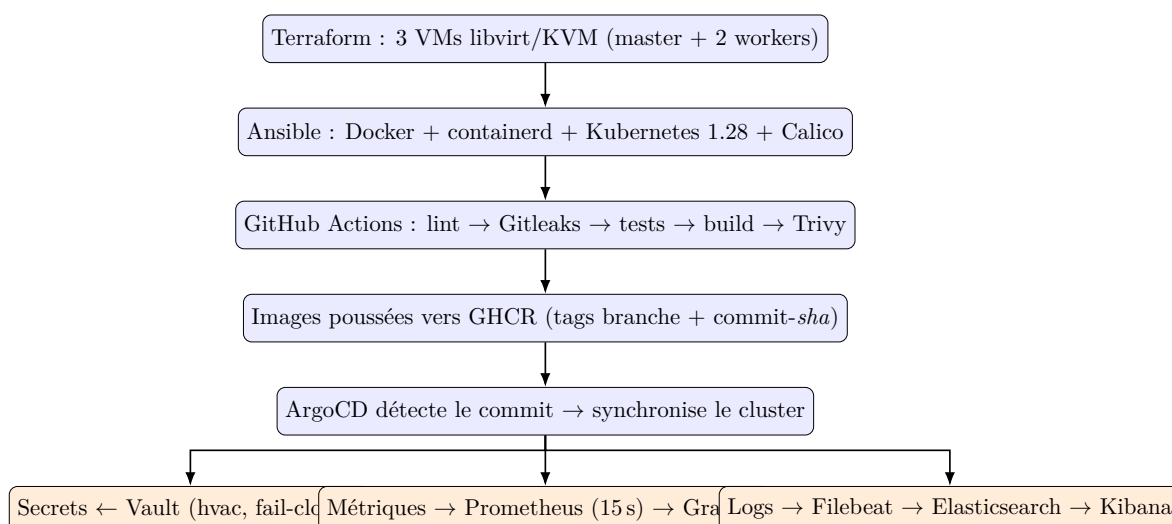
1.3 Vue d'ensemble de l'architecture

1.3.1 Les six dimensions de la plateforme

1. **Infrastructure as Code** : création automatisée et reproductible des serveurs (Terraform) ;

2. **Configuration automatisée** : installation et paramétrage des serveurs et du cluster (cloud-init, Ansible) ;
3. **Conteneurisation & orchestration** : empaquetage et exécution orchestrée (Docker, containerd, Kubernetes, Calico) ;
4. **Sécurité intégrée (DevSecOps)** : scan du code, des dépendances et des images, gestion dynamique des secrets, politiques d'admission (Gitleaks, Trivy, pip-audit, Vault, conftest, Kyverno) ;
5. **Déploiement automatisé (GitOps)** : synchronisation continue Git ↔ cluster (GitHub Actions, ArgoCD) ;
6. **Observabilité complète** : métriques et logs centralisés, alerting SLO (Prometheus, AlertManager, Grafana, ELK).

1.3.2 Flux de bout en bout



1.3.3 Inventaire rapide des technologies

Le tableau final (§9.3) recense **plus de quarante outils**, chacun détaillé dans une section dédiée : hyperviseur, IaC, configuration, runtime conteneurs, CNI, framework applicatif, bibliothèques Python, gestionnaire de secrets, registre, six outils de scan/qualité, moteur GitOps, quatre composants de monitoring et quatre composants ELK.

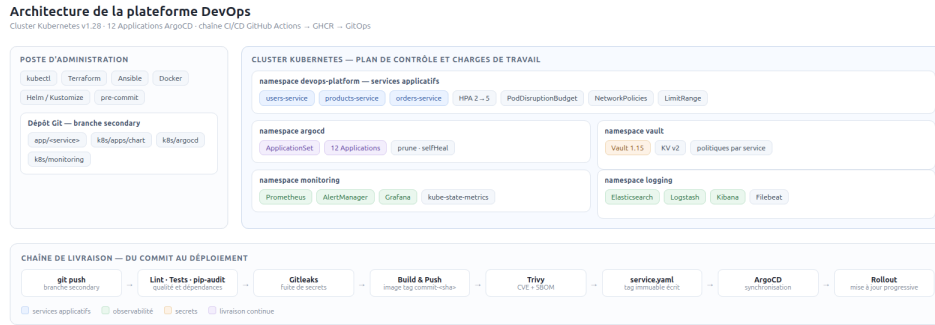


Figure 1.1. Schéma d'architecture général redessiné ou photographié depuis un tableau blanc / outil de diagramme : les 3 VMs, le cluster K8s, les namespaces (devops-platform, monitoring, vault, argocd), les flux CI et GitOps.

1.4 Fiches d'identité des outils majeurs

Tableau 1.1. Fiches d'identité des outils majeurs — vue synthétique

Outil	Version	Succès mesuré par
Terraform	~1.5 (CI 1.5.7), libvirt 0.9.8	3 domains Ready, inventaire régénéré sans divergence
Ansible	core community.general 8.0.0	+ PLAY RECAP rc=0, nœuds Ready, dpkg hold actifs
Docker containerd	/ multi-stage non-root	images <150 Mo healthy, SystemdCgroup=true partout
Kubernetes + Calico	1.28 épinglée, Calico v3.26.1	nodes Ready, netpol effectives, HPA fonctionnel
Vault + hvac	1.15.2 (dev mode), hvac 2.1.0	readyz 200, rotation sans fuite argv/disk/git
GitHub Actions ArgoCD	actions v4/v5, trivy + v0.24.0, ArgoCD v3.5.1	runs verts, Applications Synced+Healthy
Stack observabilité	Prometheus 0.74/v2.51, Grafana 11.1, ELK 8.14	targets UP, alertes SLO armées, logs Kibana

Tableau 1.3. Inventaire des namespaces

Namespace	PSA	Contenu
devops-platform	restricted	3 services (SA dédiés, HPA, PDB, netpol), quotas, Secret vault-root-token
monitoring	restricted	Prometheus, AlertManager, Grafana+sidecar, ES, Filebeat DS, Logstash, Kibana, KSM, ServiceMonitors, rules SLO, 4 dashboards CM
vault	baseline	Vault 1.15.2 dev-mode + Job setup (IPC_LOCK requis)
argocd	—	ArgoCD v3.5.1 complet + Applications
kube-system	/ —	composants cluster (hors périmètre applicatif)
calico-system		

1.5 Topologie réseau et dimensionnement

1.5.1 Réseau virtuel

Réseau libvirt dédié créé par Terraform :

- nom : `devops-platform-net`, domaine DNS local `devops.local` ;
- mode : isolé avec NAT vers l'extérieur ;
- DHCP : de `192.168.56.100` à `192.168.56.200` (les nœuds eux-mêmes sont en IP statiques hors de cette plage) ;
- sous-réseau : `192.168.56.0/24`, validé par Terraform (CIDR valide, plage RFC1918 obligatoire) ;
- relais DNS vers `1.1.1.1` et `8.8.8.8`.

Plan d'adressage des nœuds :

Nœud	IP statique	Rôle cluster	Dimensionnement
master-01	192.168.56.10	control-plane	2 vCPU / 4 Go / 20 Go qcow2
worker-01	192.168.56.11	worker	2 vCPU / 4 Go / 20 Go qcow2
worker-02	192.168.56.12	worker	2 vCPU / 4 Go / 20 Go qcow2

Tableau 1.5. Topologie par défaut (worker_count paramétrable de 1 à 32)

Les variables Terraform fixent `vm_memory_mb = 4096` avec un commentaire explicite : à 2048 Mo, la pile complète (ELK + ArgoCD + supervision) fait basculer le kubelet en `NodeStatusUnknown` sous pression mémoire : décision *justifiée par incident*. Trois

plans d'adressage coexistent sans conflit : réseau des VMs (192.168.56.0/24), CIDR pods (192.168.0.0/16), CIDR services (10.96.0.0/12).

CHAPITRE 2

Infrastructure as Code et configuration automatisée

2.1 Libvirt/KVM et cloud-init

KVM (hyperviseur noyau Linux) + libvirt (API de management standard) fournissent trois vraies VM Ubuntu Server 22.04 : type q35, disques qcow2 clonés (copie-on-write) depuis une image de base, carte réseau virtio, console série + VNC localhost. Le provider Terraform `dmacvicar/libvirt` pilote le tout déclarativement.

cloud-init (standard de facto du premier démarrage) applique dès le boot : `ssh_pwauth: false`, `disable_root: true` (clé SSH uniquement, aucun mot de passe), `fail2ban` installé, IP statique par nœud. Sudo est **temporairement large** (`NOPASSWD:ALL`) le temps que les rôles Ansible installent des paquets (une règle restreinte dès le boot ferait échouer la première tâche du playbook. Le rôle Ansible `harden_sudo`, joué explicitement en fin de course (`-tags harden`, non appliqué par défaut), repose ensuite une politique `NOPASSWD` restreinte à une liste blanche (`kubeadm`, `kubelet`, quelques `systemctl restart`)) `NOPASSWD` et non `PASSWD`, car aucun mot de passe n'existe pour ce compte : l'exiger verrouillerait définitivement le nœud hors de portée SSH.

Justification du choix technique

KVM/libvirt offre de *vraies* VM Linux (kernel réel, `systemd`, SSH) sans coût cloud. Appliquer le durcissement dès le premier boot, avant même qu'Ansible n'intervienne, garantit qu'aucune VM ne passe un seul instant dans un état faible.

2.2 Terraform — Infrastructure as Code

Terraform (HashiCorp) décrit l'infrastructure de manière déclarative en HCL : état voulu, plan prévisualisé, état détectant la dérive. Providers verrouillés : `dmacvicar/libvirt` ~> 0.9 (résolu 0.9.8), `hashicorp/local` ~> 2.5, `hashicorp/null` ~> 3.2 (correctif de capacité de volume, voir ci-dessous).

Ressources principales : `libvirt_network.platform` (NAT + DHCP), `libvirt_volume.node` (par nœud, `for_each`), `libvirt_cloudinit_disk.init`, `libvirt_domain.node` (VM q35, virtio), `local_file.ansible_inventory` (génère l'inventaire Ansible : Terraform reste seule source de vérité, il ne peut jamais diverger de l'infrastructure réelle). Variables principales : `worker_count` 2 (soit 3 nœuds), `vm_vcpu` 2, `vm_memory_mb` 4096, `disk_size_gb` 20.

Bug de provider corrigé : `libvirt_volume` créé par clonage d'URL ignore la capacité déclarée et hérite de la taille de l'image source. Corrigé par un `null_resource.fix_volume_capacity` (`local-exec virsh vol-resize`) explicitement ordonné avant le domaine (`depends_on`) : validé sur plusieurs créations complètes de VMs.

Backend local actif (`terraform.tfstate`, gitignored) ; le contrat de production (S3 + verrous DynamoDB) est prêt en commentaire, motivé par le risque d'applis concurrents corrompant un état non verrouillé.

Justification du choix technique

Terraform est le standard industriel de l'IaC : l'épinglage ~> 1.5 privilégie la reproductibilité (la CI rejoue exactement 1.5.7) plutôt que l'actualité.

```
Terraform — Plan d'exécution 16 ressources à créer, aucune destruction 28/08/2026 17:45

devops@homelab:~/rapports$ cd terraform && terraform plan -no-color -input=false 2>&1 | tail -20
# terraform_data.ssh_key_guard will be created
+ resource "terraform_data" "ssh_key_guard" {
+   id      = (known after apply)
+   input   = (sensitive value)
+   output  = (known after apply)
+ }

Plan: 16 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ansible_inventory_content = (sensitive value)
+ ansible_inventory_file    = (sensitive value)
+ master_ip                 = (sensitive value)
+ node_ips                  = (sensitive value)
+ worker_ips                = (sensitive value)

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.
```

Figure 2.1. `terraform plan` : create des domaines/volumes/cloudinit disks avec le résumé final « Plan: N to add, 0 to change, 0 to destroy ».

2.3 Ansible — configuration automatisée

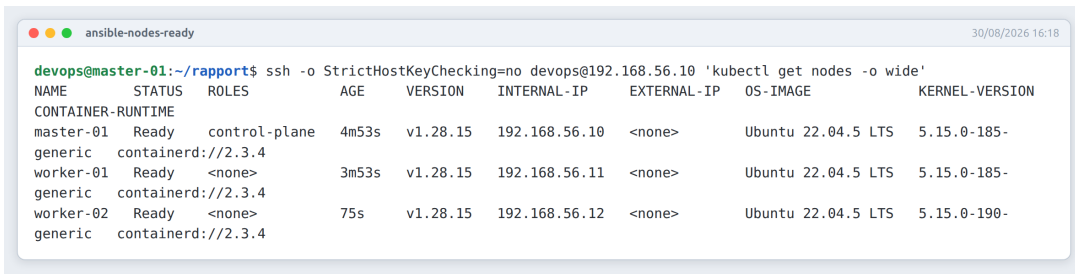
Ansible configure les VMs par SSH sans agent, de façon idempotente, en rôles réutilisables. Cinq plays successifs dans `playbook.yml` : `docker`, `k8s_common`, `k8s_master` (hosts masters), `k8s_worker` (hosts workers, `serial: 1` : jointure séquentielle), et `k8s_reset` (destructif, double opt-in : `tag never et -e`

`reset_confirmed=true`). Valeurs cluster centrales : `k8s_version: "1.28"`, Calico v3.26.1, pod CIDR 192.168.0.0/16, service CIDR 10.96.0.0/12.

- **docker** : dépôt officiel, `docker-ce/containerd.io`, bascule `SystemdCgroup = true` dans `/etc/containerd/config.toml` (obligatoire pour kubeadm) ;
- **k8s_common** : swap désactivé, modules `overlay/br_netfilter`, `sysctls forwarding`, `kubelet/kubeadm/kubectl` épinglés 1.28.* et verrouillés par `dpkg hold` ;
- **k8s_master** : `kubeadm init` (addons `coredns/kube-proxy` différés), Calico via l'opérateur Tigera en *fail-closed* (cluster jamais déclaré prêt tant que Calico n'est pas Available), join command généré avec `no_log: true` et stocké en 0600 ;
- **k8s_worker** : jointure via module `command:` (pas `shell:`, anti-injection), idempotente (`creates: /etc/kubernetes/kubelet.conf`).

Justification du choix technique

Ansible est agentless : seul SSH suffit, cohérent avec des VMs recréables à volonté par Terraform. Le double verrou version épinglée + `dpkg hold` interdit toute dérive de version au fil des upgrades système ; le `no_log` sur le join token et le *fail-closed* Calico traduisent une pratique acquise par incidents réels.



```
devops@master-01:~/rapports$ ssh -o StrictHostKeyChecking=no devops@192.168.56.10 'kubectl get nodes -o wide'
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION
master-01	Ready	control-plane	4m53s	v1.28.15	192.168.56.10	<none>	Ubuntu 22.04.5 LTS	5.15.0-185-
generic	containerd://2.3.4							
worker-01	Ready	<none>	3m53s	v1.28.15	192.168.56.11	<none>	Ubuntu 22.04.5 LTS	5.15.0-185-
generic	containerd://2.3.4							
worker-02	Ready	<none>	75s	v1.28.15	192.168.56.12	<none>	Ubuntu 22.04.5 LTS	5.15.0-190-
generic	containerd://2.3.4							

Figure 2.2. `kubectl get nodes -o wide` juste après le run : 3 nœuds Ready v1.28.x.

CHAPITRE 3

Conteneurisation des services et orchestration Kubernetes

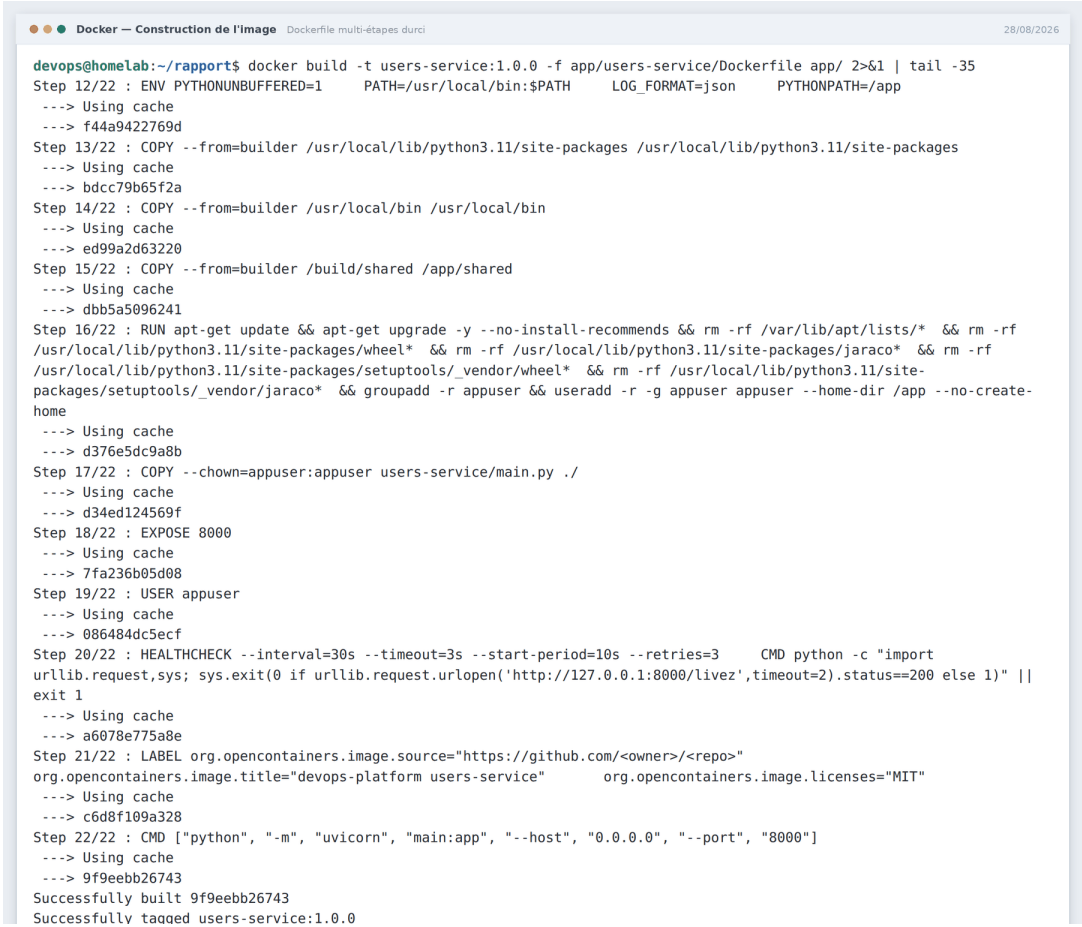
3.1 Docker — construction des images

Docker sert exclusivement à **construire** les images OCI ; l'exécution en cluster est assurée par containerd. Les trois services partagent le même modèle de Dockerfile **multi-stage durci** : stage **builder** installe les dépendances (`pip install ./shared`) puis le stage final ne copie que les site-packages et le code, sans pip ni caches ni wheels.

- **non-root** : utilisateur **appuser** dédié, aligné avec **runAsNonRoot** K8s ;
- **HEALTHCHECK** natif sur **/livez**, défense en profondeur complétée par les probes K8s ;
- **labels OCI** (source, titre, licence) pour la traçabilité ;
- contexte de build = **app/** (jamais le dossier du service seul) afin que **shared/** soit importable comme paquet racine ; **.dockerignore** strict exclut **.git**, **tfstate**, **k8s/**, **terraform/**.

Justification du choix technique

Le multi-stage est le standard des images production : il sépare compilation et exécution. Combiné à non-root + HEALTHCHECK + labels OCI, chaque image est conforme aux exigences des registres modernes et directement scannable par Trivy sans bruit parasite.



```
devops@homeLab:~/rapports$ docker build -t users-service:1.0.0 -f app/users-service/Dockerfile app/ 2>&1 | tail -35
Step 12/22 : ENV PYTHONUNBUFFERED=1 PATH=/usr/local/bin:$PATH LOG_FORMAT=json PYTHONPATH=/app
--> Using cache
--> f44a9422769d
Step 13/22 : COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
--> Using cache
--> bdcc79b65f2a
Step 14/22 : COPY --from=builder /usr/local/bin /usr/local/bin
--> Using cache
--> ed99a2d63220
Step 15/22 : COPY --from=builder /build/shared /app/shared
--> Using cache
--> dbb5a5096241
Step 16/22 : RUN apt-get update && apt-get upgrade -y --no-install-recommends && rm -rf /var/lib/apt/lists/* && rm -rf
/usr/local/lib/python3.11/site-packages/wheel* && rm -rf /usr/local/lib/python3.11/site-packages/jaraco* && rm -rf
/usr/local/lib/python3.11/site-packages/setuptools/_vendor/wheel* && rm -rf /usr/local/lib/python3.11/site-
packages/setuptools/_vendor/jaraco* && groupadd -r appuser && useradd -r -g appuser appuser --home-dir /app --no-create-
home
--> Using cache
--> d376e5dc9a8b
Step 17/22 : COPY --chown=appuser:appuser users-service/main.py ./
--> Using cache
--> d34ed124569f
Step 18/22 : EXPOSE 8000
--> Using cache
--> 7fa236b05d08
Step 19/22 : USER appuser
--> Using cache
--> 086484dc5ecf
Step 20/22 : HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 CMD python -c "import
urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://127.0.0.1:8000/livez',timeout=2).status==200 else 1)" ||
exit 1
--> Using cache
--> a6078e775a8e
Step 21/22 : LABEL org.opencontainers.image.source="https://github.com/<owner>/<repo>"
org.opencontainers.image.title="devops-platform users-service" org.opencontainers.image.licenses="MIT"
--> Using cache
--> c6d8f109a328
Step 22/22 : CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
--> Using cache
--> 9f9eebb26743
Successfully built 9f9eebb26743
Successfully tagged users-service:1.0.0
```

Figure 3.1. docker build d'un service : couches builder puis export final visibles.

3.2 containerd et Kubernetes 1.28 — socle du cluster

containerd (runtime CRI, promu CNCF, extrait de Docker) exécute tous les Pods de la plateforme : pull d'images GHCR, exécution via runc. Sa configuration est corrigée pour activer `SystemdCgroup = true`, alignant containerd sur le kubelet 1.28 : sans cela, un dual-cgrouping cgroupfs/systemd provoque des OOMKills fantômes et évictions injustifiées.

kubeadm amorce le control-plane avec des paramètres explicites (adresse API, CIDR pods/services, socket CRI). Les phases add-on (CoreDNS, kube-proxy) sont volontairement **différées** après coup pour éviter un CrashLoop de CoreDNS avant que le CNI ne fournisse le réseau. Le réseau des pods est assuré par **Calico v3.26.1** (opérateur Tigera) qui applique l'intégralité des NetworkPolicies ingress/egress : toute la stratégie réseau (default-deny + whitelist) en dépend. L'installation est gardée stricte : un demi-cluster ne se déclare jamais prêt.

Justification du choix technique

Depuis la suppression de dockershim (K8s 1.24), containerd est le chemin recommandé par kubeadm. Calico combine performance de routage L3 et le support NetworkPolicy le plus complet de l'écosystème : un CNI sans egress policies aurait invalidé une section entière du modèle de sécurité.



```
devops@master-01:~/rapports$ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
devops-platform-control-plane       Ready    control-plane   13d   v1.36.1   172.18.0.2    <none>        Debian GNU/Linux 13
(trixie) 7.0.0-30-generic (amd64)   containerd://2.3.1
```

Figure 3.2. `kubectl get nodes -o wide` : INTERNAL-IP 192.168.56.10–12, OS Ubuntu 22.04, containerd://x.y.z.

Pour chacun des trois services, le bundle Kustomize produit environ **30 objets** (Deployment, Service, ServiceMonitor, HPA, PDB, ServiceAccount, NetworkPolicies partagées, Secret consommé) : chaque ligne synchronisée par ArgoCD.

3.3 Manifests applicatifs — Kustomize

Kustomize personnalise des bundles YAML sans templating : un **base** décrit l'application générique, des overlays (`dev/staging/prod`) patchent les écarts par environnement. Natif dans `kubectl` et première classe citoyenne chez ArgoCD.

- **Namespace durci** : Pod Security Admission `restricted`, `LimitRange` (défauts 250m/256Mi, max 4 CPU/2Gi), `ResourceQuota` (4 CPU/4Gi requests, 20 pods max) ;
- **Pods** : `runAsNonRoot`, `readOnlyRootFilesystem`, `capabilities: drop ALL`, `startupProbe/livenessProbe` sur `/livez`, `readinessProbe` sur `/readyz` : un pod vivant mais privé de Vault sort du Service plutôt que de servir dégradé ;
- **Autoscaling** : HPA v2 min 2 / max 5, cibles CPU 70 %/mémoire 80 %, scale-up rapide (30 s) / scale-down prudent (300 s, anti-flapping) ; PDB minAvailable 1 (2 en prod) ;
- **RBAC** : ServiceAccounts dédiés, `automountServiceAccountToken: false` (aucun appel API K8s nécessaire) ;
- **NetworkPolicies** : `default-deny` + liste blanche (egress Vault, egress DNS, ingress Prometheus, intra-namespace) ;
- **Overlays** : `dev` réplicas=1, `staging` namespace dédié, `prod` réplicas=3/PDB=2/images épinglées par digest.

Justification du choix technique

Sans default-deny, toute compromission d'un Pod ouvre latéralement tout le cluster ; le modèle liste blanche exprime exactement les flux métier légitimes et rien d'autre, appliqué au niveau noyau par Calico. Kustomize patche sans langage de template : chaque environnement ne déclare que ses écarts.

```
devops@master-01:~/rapports$ ./scripts/capture/demo/netpol-proof-kind.sh
# Pod du meme namespace, etiquette autorisee :
code=200
# Pod d'un autre namespace, sans etiquette :
code=000
```

Figure 3.3. Preuve default-deny : un Pod sans label ne peut joindre users-service, un Pod du namespace oui (`kubectl run test`).

```
devops@master-01:~/rapports$ kubectl get hpa -n devops-platform
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
catalog-items	Deployment/catalog-items	cpu: 1%/70%, memory: 28%/80%	2	5	2	13d
inventory-service	Deployment/inventory-service	cpu: 1%/70%, memory: 28%/80%	2	5	2	13d
orders-service	Deployment/orders-service	cpu: 1%/70%, memory: 27%/80%	2	5	2	13d
products-service	Deployment/products-service	cpu: 1%/70%, memory: 27%/80%	2	5	2	13d
users-service	Deployment/users-service	cpu: 1%/70%, memory: 27%/80%	2	5	2	13d

Figure 3.4. `kubectl get hpa` : targets CPU/mémoire, min/max, réplicas courants.

CHAPITRE 4

Architecture applicative et gestion dynamique des secrets

4.1 FastAPI + Uvicorn — le socle applicatif

Trois microservices Python FastAPI 0.139.0 / Uvicorn (structure identique : `main.py`, `requirements.txt`, `Dockerfile`, port 8000), chacun avec ses propres secrets Vault (ex. `users-service` : `DATABASE_URL`, `JWT_SECRET_KEY`). Endpoints communs : `/` (carte d'identité), `/livez`, `/readyz`, `/metrics`.

Points d'analyse du code réel :

- **chargement au démarrage** : les secrets sont résolus à l'import du module : un problème Vault devient un crash immédiat et explicite, pas une erreur 500 tardive ;
- **fail-closed en prod, repli assumé en dev** : en dev la base tombe sur sqlite mémoire et la clé JWT est éphémère (`secrets.token_hex(32)`), chaque repli loggé en warning avec événement structuré ; en production, `SystemExit` si un secret manque ;
- **séparation livez/readyz** : la readiness dépend explicitement de Vault (503 sinon) : le pod sort du Service au lieu de servir dégradé.

Justification du choix technique

FastAPI combine validation Pydantic, OpenAPI auto, typage moderne et performances ASGI. Trois services quasi-identiques mettent en évidence ce qui compte : la plateforme transverse (CI, secrets, monitoring) identique pour tous.

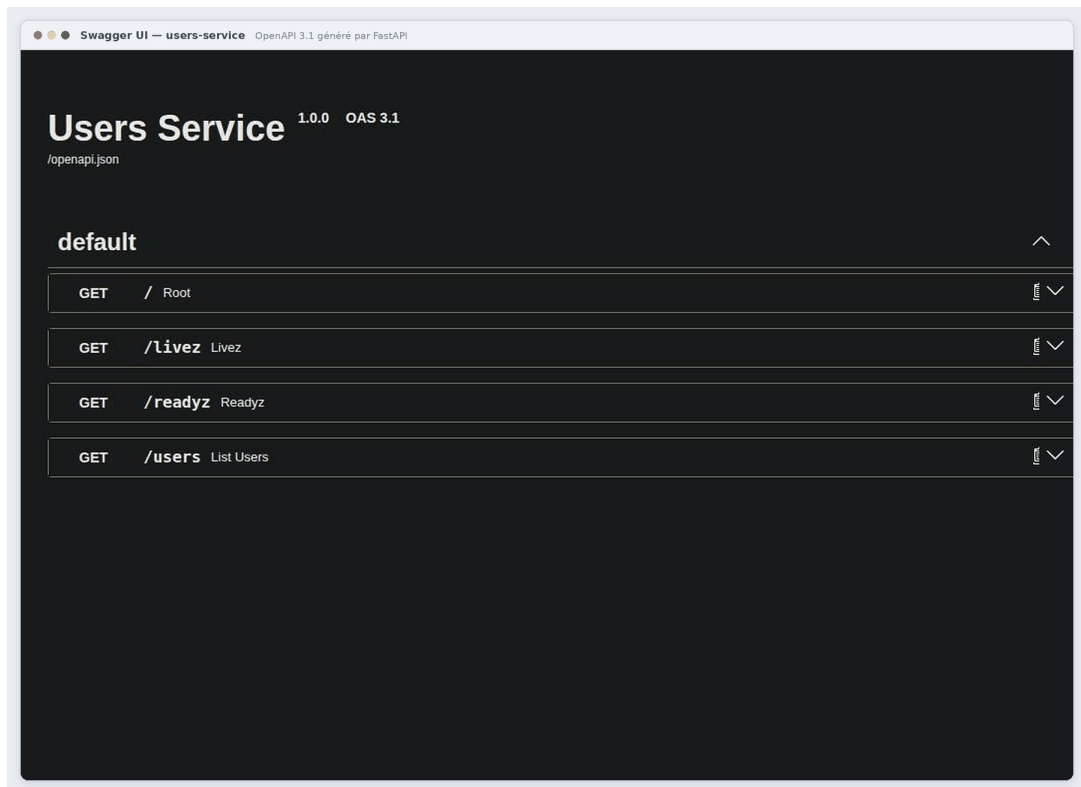


Figure 4.1. `kubectl port-forward svc/users-service 8080:80` puis navigateur sur `/docs` : Swagger UI OpenAPI automatique.

4.2 Bibliothèque partagée shared/ et instrumentation

Paquet Python installé dans chaque image (`pip install ./shared`) : garantit que les trois services partagent exactement la même politique de secrets, le même format de logs et la même configuration. Dépendances épinglées : `hvac==2.1.0` (client Vault KV-v2), `prometheus-fastapi-instrumentator==6.1.0`, `python-json-logger==2.0.7`.

`vault_client.py` implémente le contrat fail-closed : ordre de résolution du token (Vault Agent Injector → `VAULT_TOKEN` → `VAULT_DEV_ROOT_TOKEN_ID`), lecture du chemin KV-v2 `secret/data/devops-platform/<SERVICE_NAME>`, cache processus (`lru_cache`), et surtout : **tout repli (env, défaut) est loggé en warning** : un fallback silencieux est impossible, l'absence de secret et d'override lève `SecretUnavailable`. `log_config.py` structure les logs en JSON (`stdout`) ou format humain en dev.

Chaque service expose `/metrics` via une configuration commune de l'Instrumentator (codes groupés, routes non déclarées ignorées, `/livez//readyz//metrics` exclus) : ces métriques RED alimentent directement les règles SLO (voir Observabilité).

Justification du choix technique

Centraliser secrets/logging/config dans un paquet partagé garantit un comportement identique des trois services et un correctif unique. hvac est le client Vault Python de référence ; une ligne d'Instrumentator donne les métriques RED normalisées là où un câblage manuel aurait dupliqué du middleware fragile dans les trois services.

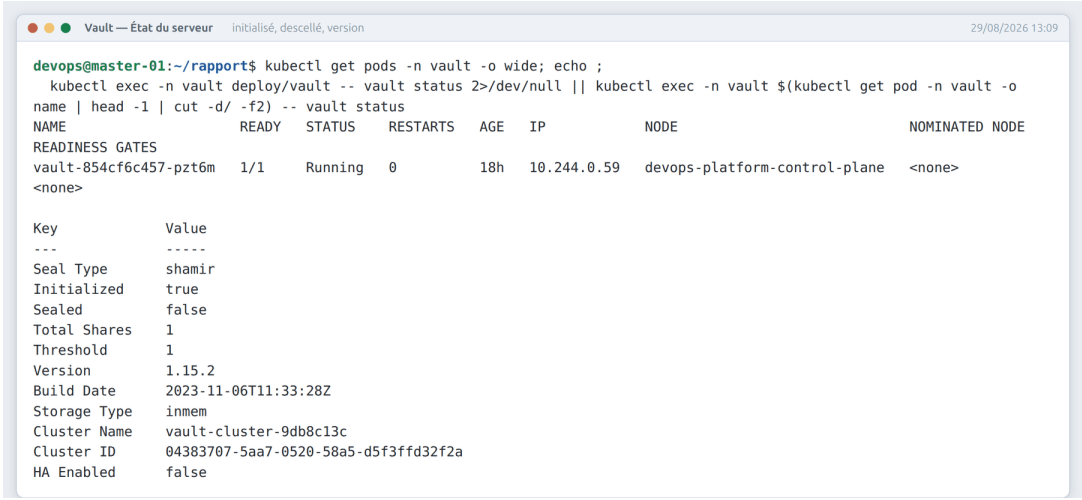
4.3 HashiCorp Vault — gestion dynamique des secrets

HashiCorp Vault **1.15.2** centralise les secrets : stockage chiffré, distribution contrôlée, rotation, révocation, audit. Déploiement volontairement en **mode dev** (un réplica, stockage mémoire, auto-unseal) : namespace au profil « baseline » plutôt que « restricted » car Vault requiert la capacité noyau `IPC_LOCK`. Ce choix de lab démontre dès aujourd'hui toute la mécanique applicative (KV-v2, politiques, auth Kubernetes, rotation) sans porter la complexité de production ; le chemin de montée en gamme (raft répliqué, TLS interne, unseal Shamir, agent injector) est tracé en feuille de route.

Un Job d'amorçage (`vault-setup`) monte le moteur KV-v2, active l'auth Kubernetes, applique une politique par service en **moindre privilège** : `read` sur la donnée, `read+list` sur les métadonnées : jamais de `delete` ni `write` pour l'application (la suppression reste opérateur-only). Le token root est distribué hors bande par un script qui génère une valeur aléatoire, la passe **par stdin** (jamais en argv ni sur disque) et permet une rotation idempotente.

Justification du choix technique

Vault résout le vrai problème des secrets : distribution dynamique, auditabilité, révocation, TTL courts : là où les Secrets Kubernetes seuls sont du base64 non chiffré. Le fail-closed applicatif transforme « Vault indisponible » en incident visible dès le readiness probe plutôt qu'en fuite silencieuse.



The terminal window shows the execution of `kubectl get pods -n vault -o wide; echo ; kubectl exec -n vault deploy/vault -- vault status 2>/dev/null || kubectl exec -n vault $(kubectl get pod -n vault -o name | head -1 | cut -d/ -f2) -- vault status`. The output displays the pod status and the Vault configuration.

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
vault-854cf6c457-pzt6m	1/1	Running	0	18h	10.244.0.59	devops-platform-control-plane	<none>

Key	Value
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	1
Threshold	1
Version	1.15.2
Build Date	2023-11-06T11:33:28Z
Storage Type	inmem
Cluster Name	vault-cluster-9db8c13c
Cluster ID	04383707-5aa7-0520-58a5-d5f3ffd32f2a
HA Enabled	false

Figure 4.2. `kubectl get pods -n vault + vault status` exec dans le pod : initialized true, sealed false.

CHAPITRE 5

Chaîne d'intégration continue et sécurité intégrée (DevSecOps)

5.1 GitHub Actions — pipeline CI/CD

Workflow unique `.github/workflows/ci-cd.yml` déclenché sur push (filtré par chemins), `pull_request`, cron nocturne (drift CVE) et `workflow_dispatch` (seule porte d'entrée du deploy production). Concurrency par ref (`cancel-in-progress`), permissions minimales par job.

```
lint ----> test --> build --> trivy-scan ----> deploy (manuel)
    | |
gitleaks+--> terraform-validate -----+
load-test (schedule uniquement, independant)
```

Listing 5.1. Ordre et dépendances des jobs

- **lint** : ruff 0.1.9, yamllint 1.33.0, `terraform fmt -check`, kubeconform v0.6.4 (`-strict`, Kubernetes 1.28.0), conftest en audit ;
- **test** : `pip-audit -strict` sur les 4 requirements, pytest, import réel de chaque `main.py`, builds Docker de fumée ;
- **build** : matrice des 3 services, buildx + cache GHA, tags branche/commit-sha/latest, attestations provenance+SBOM, image épinglée par digest ;
- **trivy-scan** : SARIF publié (Security tab), gate bloquant CRITICAL/HIGH (exit-code 1), SBOM SPDX en artefact ;
- **deploy** (manuel, environment production) : kustomize edit set image par digest, preflight kubeconform, application server-side, attente rollouts 120s, smoke-test /metrics.

Justification du choix technique

GitHub Actions vit là où vit le code : zéro infrastructure CI à maintenir, matrice native, marketplace riche. Les tags immuables branche+sha et l'absence de `:latest`

mutable hors branche par défaut rendent chaque déploiement retraçable jusqu'au commit exact. Le deploy n'est jamais automatique : l'humain reste dans la boucle pour la prod.

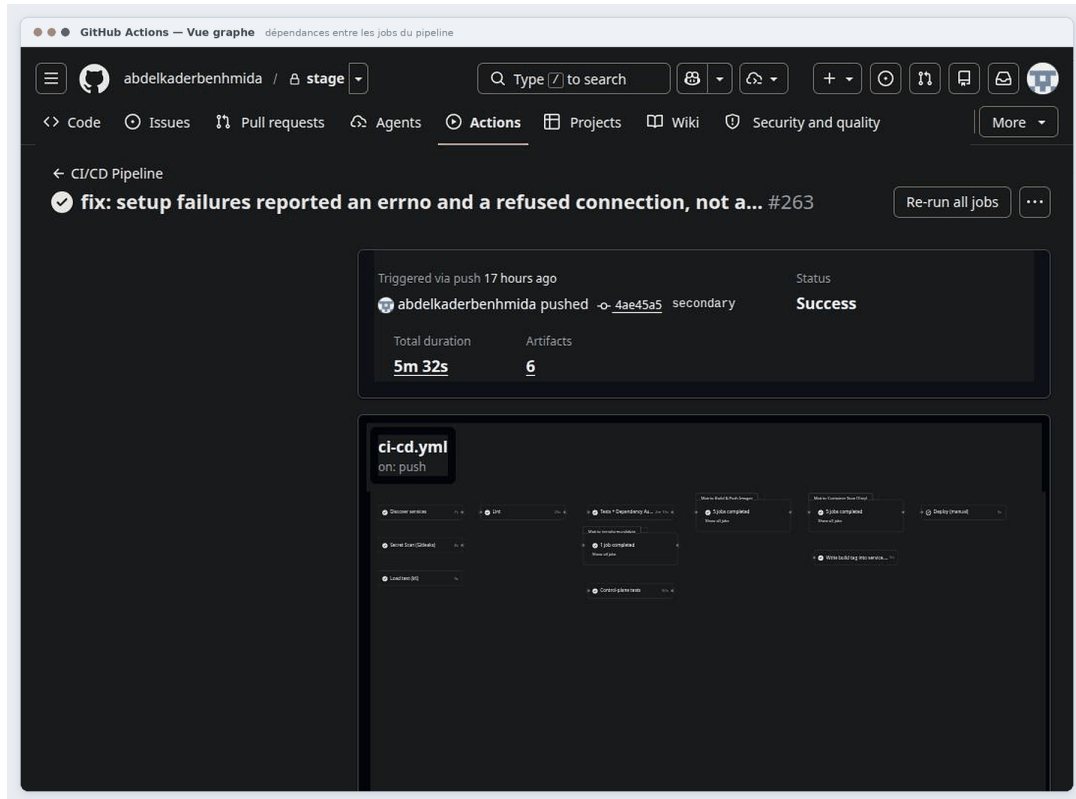


Figure 5.1. Vue graphe d'un run : jobs lint/gitleaks/test/build/trivy-scan/terraform-validate et leurs dépendances.

5.2 Gitleaks — détection de secrets

Gitleaks scanne l'historique Git et le working tree à la recherche de motifs de secrets (clés API, tokens, privés). Double enforcement : hook pre-commit (gitleaks v8.18.0) avant chaque push, et job CI bloquant sur l'historique complet (`fetch-depth: 0`). Configuration `.gitleaks.toml` : règles par défaut étendues + allowlist resserrée (chemins `terraform.tfstate`, placeholders documentés uniquement : l'ancienne allowlist trop large, type `.*md`, a été retirée).

Justification du choix technique

La fuite de secret la plus fréquente reste le commit accidentel. Le double enforcement (pre-commit + CI sur toute l'historique) couvre les deux fenêtres.

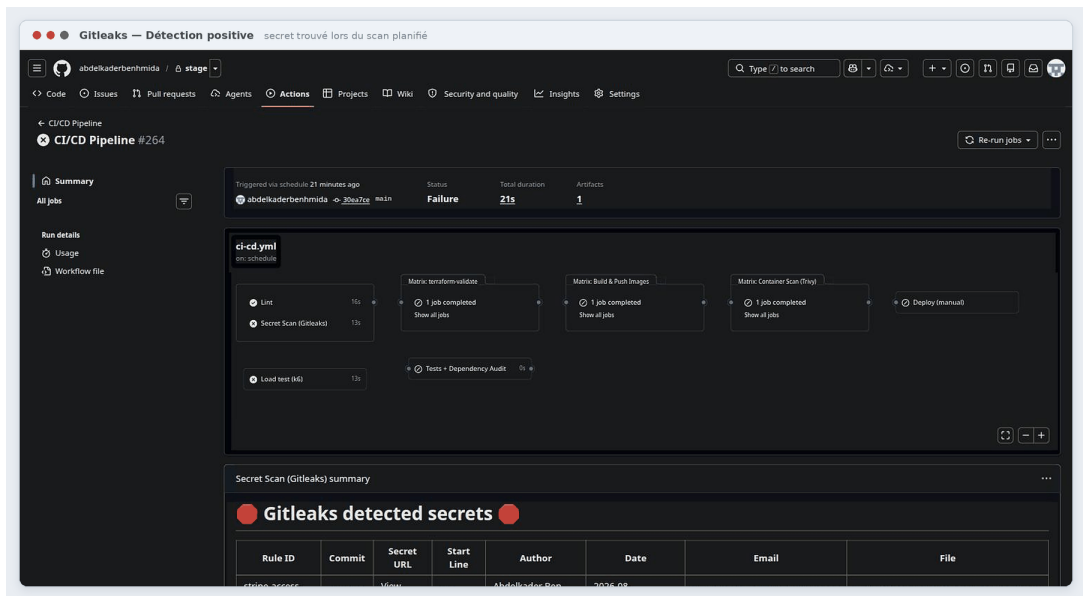


Figure 5.2. Test positif contrôlé : commit d'un faux token AWS puis run gitleaks le détectant (puis revert).

5.3 Trivy, pip-audit et pre-commit

Trivy (Aqua Security, scanner unifié CNCF) fait trois passes par image : SARIF vers l'onglet Security GitHub, gate bloquant CRITICAL/HIGH (`exit-code 1`, `ignore-unfixed`), SBOM SPDX archivé 30 jours : les mêmes seuils sont rejoués localement par `validate-platform.sh` et `validate-security.sh`. **pip-audit** (auditeur officiel PyPA) tourne en mode `-strict` sur les quatre requirements ; le job nocturne le rejoue pour détecter le drift supply-chain (une CVE publiée après merge fait échouer le run de 3h00).

pre-commit exécute localement les mêmes outils que la CI (onze hooks : `ruff`, `yamllint`, `gitleaks v8.18.0`, `terraform_fmt/validate`, `detect-private-key...`), éliminant le va-et-vient « push → échec CI → fix ».

Justification du choix technique

Trivy couvre en un outil ce qui demanderait plusieurs scanners ; bloquer uniquement sur les CVE corrigeables évite les pipelines morts-nés sur des failles sans correctif. La parité stricte outils/versions entre pre-commit et CI supprime la divergence classique local/pipeline.

```
devops@homelab:~/rapport$ trivy image --scanners vuln --severity CRITICAL,HIGH --exit-code 1 --no-progress python:3.9-slim
2>&1 | tail -18; echo "exit code: ${PIPESTATUS[0]}"
| jaraco.context (METADATA) | CVE-2026-23949 | HIGH | fixed | 5.3.0 | 6.1.0 | jaraco.context:
jaraco.context: Path traversal via malicious
| | | | | | | tar archives
| | | | | | |
https://avd.aquasec.com/nvd/cve-2026-23949
| | | | | | |
| wheel (METADATA) | CVE-2026-24049 | | | 0.45.1 | 0.46.2 | wheel: wheel:
Privilege Escalation or Arbitrary Code | | | | |
malicious wheel file... | | | | | Execution via
| | | | | |
https://avd.aquasec.com/nvd/cve-2026-24049
| | | | | |
| | | | | |
| | | | | |
| | | | | |
```

Figure 5.3. Sortie bloquante : image volontairement vulnérable scannée avec `-exit-code 1` : code retour 1 visible.

5.4 Lint et validation des manifests

Ruff 0.1.9 (linter Python, remplace flake8/isort/pyflakes) et **yamllint** 1.33.0 tournent identiquement en hook et en CI. **kubeconform** v0.6.4 valide hors ligne contre les schémas Kubernetes 1.28.0 (**-strict**), joué aussi en préflight du deploy (**kustomize build | kubeconform**) (aucun manifest invalide n'atteint le cluster. **confest/Rego** (**security.rego**) encode trois refus en policy-as-code : **readOnlyRootFilesystem**, **drop ALL capabilities**, **runAsNonRoot**) mode **audit** (non bloquant) actuellement. **Kyverno** ajoute deux ClusterPolicies au niveau admission (digest-pin obligatoire, security context restreint), également en Audit.

Justification du choix technique

Policy-as-code transforme les bonnes pratiques en vérifications exécutables. Le duo conftest (CI, avant déploiement) + Kyverno (cluster, à l'admission) couvre les deux instants où une mauvaise config peut entrer. Commencer en audit est la bonne trajectoire industrielle : mesurer le bruit avant de bloquer.

CHAPITRE 6

Déploiement continu selon le modèle GitOps

6.1 ArgoCD — déploiement déclaratif continu

ArgoCD (CNCF) implémente le GitOps : le cluster *tire* son état depuis Git au lieu de le recevoir d'une CI qui pousse. Installation **v3.5.1** via kustomize remote (`core-install`), elle-même une Application auto-gérée (sync wave `-100`). L'AppProject `devops-platform` borne les droits (sourceRepos, destinations, whitelist cluster-level minimale).

Tableau 6.1. ArgoCD — Douze Applications — cartographie

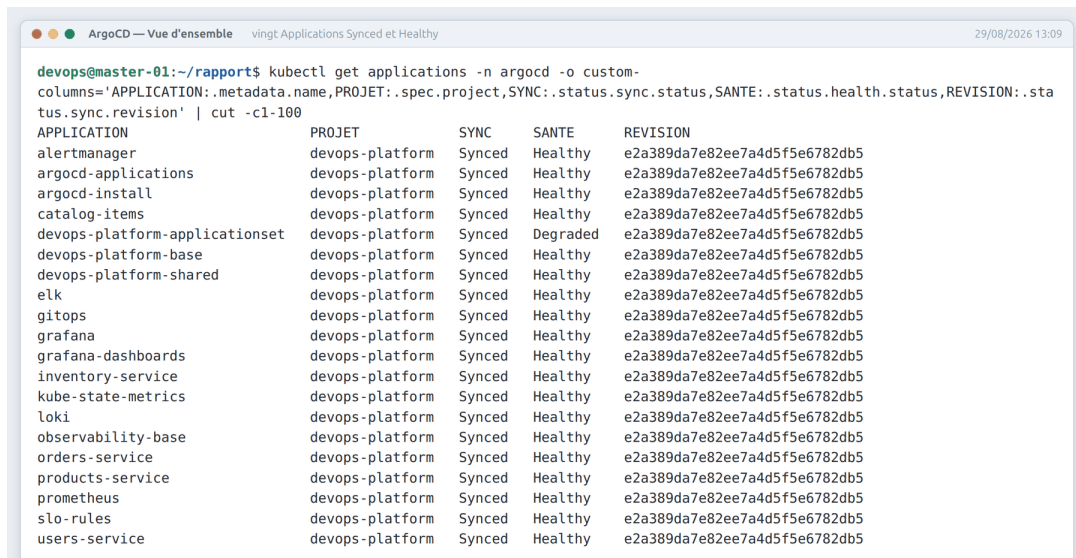
Application	Chemin Git	Wave	Destination
argocd-install	k8s/argocd/install	-100	ns argocd
observability-base	k8s/monitoring/base	5	ns monitoring
prometheus	k8s/monitoring/prometheus	15	ns monitoring
alertmanager	k8s/monitoring/alertmanager	20	ns monitoring
elk	k8s/monitoring/elk	20	ns monitoring
slo-rules	k8s/monitoring/alertmanager	20	ns monitoring
grafana	k8s/monitoring/grafana	25	ns monitoring
grafana-dashboards	k8s/monitoring/grafana/dashboards		ns monitoring
devops-platform-base	k8s/apps/base	—	ns devops-platform
users-service	k8s/apps/users	—	ns devops-platform
products-service	k8s/apps/products	—	ns devops-platform
orders-service	k8s/apps/orders	—	ns devops-platform

Politique commune aux douze Applications : **automated** + **prune** + **selfHeal** (tout commit s'applique automatiquement, toute ressource retirée de Git est purgée, tout `kubectl` edit manuel est annulé au cycle suivant) ; **ServerSideApply** +

CreateNamespace + PruneLast ; ordre garanti par des **sync waves** (ArgoCD –100, socle monitoring 5, Prometheus 15, AlertManager/ELK/SLO 20, Grafana 25 : Grafana ne démarre jamais avant que sa datasource Prometheus existe).

Justification du choix technique

Le GitOps inverse le sens du déploiement : toute dérive est corrigée par selfHeal, tout rollback est un **git revert**, l'état complet du système est auditable dans l'historique Git.



```
devops@master-01:~/rapport$ kubectl get applications -n argocd -o custom-
columns='APPLICATION:.metadata.name,PROJET:.spec.project,SYNC:.status.sync.status,SANTE:.status.health.status,REVISION:.sta
tus.sync.revision' | cut -c1-100
```

APPLICATION	PROJET	SYNC	SANTE	REVISION
alertmanager	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
argocd-applications	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
argocd-install	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
catalog-items	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
devops-platform-applicationset	devops-platform	Synced	Degraded	e2a389da7e82ee7a4d5f5e6782db5
devops-platform-base	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
devops-platform-shared	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
elk	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
gitops	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
grafana	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
grafana-dashboards	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
inventory-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
kube-state-metrics	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
loki	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
observability-base	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
orders-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
products-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
prometheus	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
slo-rules	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5
users-service	devops-platform	Synced	Healthy	e2a389da7e82ee7a4d5f5e6782db5

Figure 6.1. UI ArgoCD : tuiles des 12 Applications toutes Synced/Healthy.

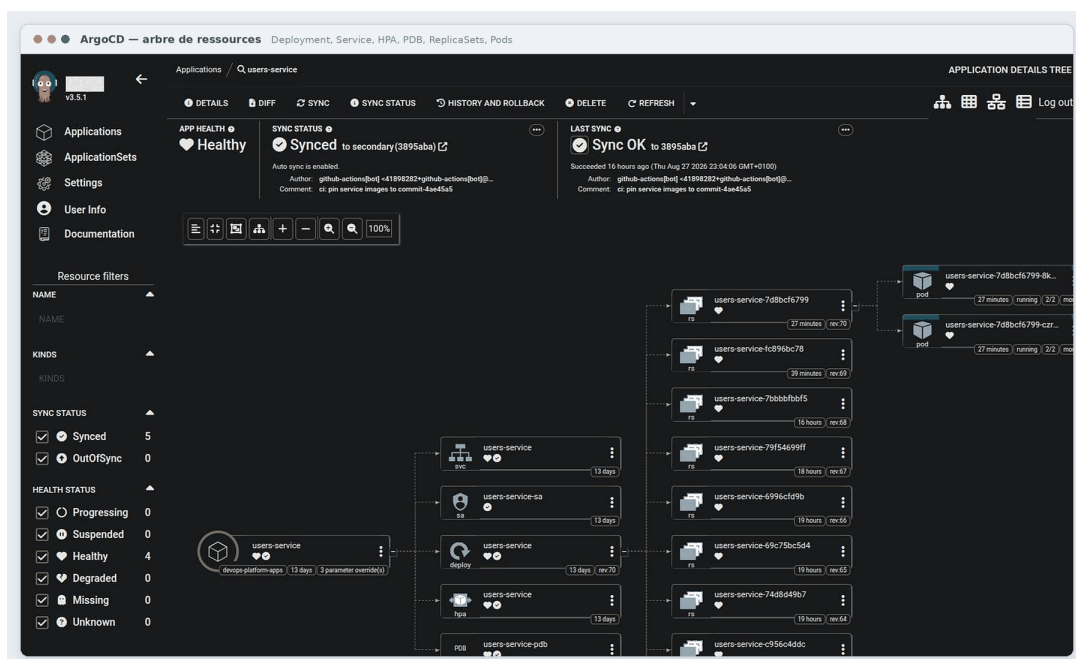


Figure 6.2. Arbre de ressources d'une Application monitoring (prometheus) avec statuts de santé.

CHAPITRE 7

Observabilité de la plateforme et objectifs de niveau de service

7.1 Prometheus Operator, kube-state-metrics et kubelet

Prometheus est le standard des métriques cloud-native (CNCF) : collecte pull par scraping HTTP, PromQL. Le Prometheus Operator (v0.74.0 + 8 CRDs) transforme sa configuration en ressources Kubernetes déclaratives ; instance Prometheus v2.51.0, rétention 15 jours (cap 8GiB), scrape/évaluation 15 s, PVC 10 GiB. Chaque service publie son propre **ServiceMonitor** (namespace monitoring, path `/metrics`, intervalle 15 s) : ajouter un service supervisé se résume à poser un ServiceMonitor, ce que GitOps synchronise sans intervention.

kube-state-metrics v2.10.1 expose l'état des objets K8s (réplicas, statut HPA, raisons de terminaison) : source indispensable des alertes `PodEvictionOngoing` et `HPAPinnedAtMaxReplicas`. Le **scrape kubelet/cadvisor** (Service headless + Endpoints explicites, scheme https) fournit CPU/mémoire/réseau par conteneur, matière première de `ContainerMemoryRatioHigh`.

Justification du choix technique

L'Operator rend la configuration de Prometheus native Kubernetes : plus d'edits de `prometheus.yml` hors cluster. L'intervalle 15 s offre assez de granularité pour les SLO tout en restant raisonnable pour l'homelab.

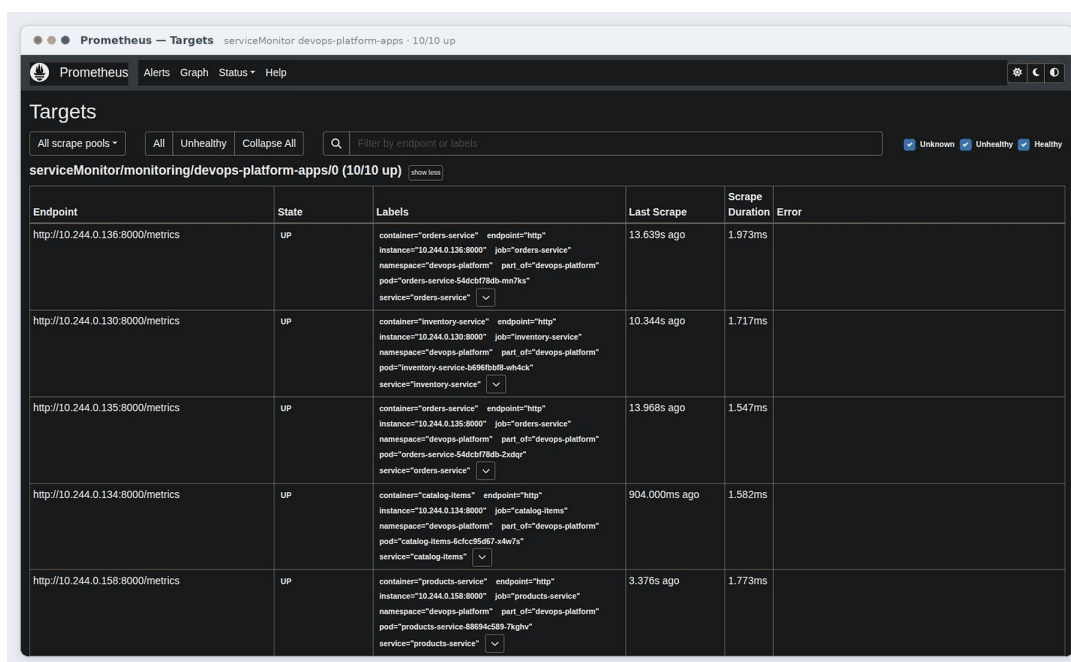


Figure 7.1. UI Prometheus (kubectl port-forward svc/prometheus 9090) : Status → Targets, tous les jobs UP.

7.2 Carnet de requêtes PromQL et AlertManager

Un carnet versionné de requêtes évite la réinvention sous stress : RPS par service (`sum by(job)(rate(http_requests_total[5m]))`), taux d'erreur 5xx, latence p95/p99 (`histogram_quantile`), ratio mémoire par conteneur, OOMKills des dernières 24h, état HPA courant/min/max, et une recording rule de disponibilité glissante 30 jours (`service:availability_30d:ratio`) pré-calculée toutes les 5 minutes pour garder l'évaluation des alertes bon marché.

Contrainte réelle de dimensionnement : sur les VMs 4 Go de l'homelab, le budget mémoire est serré (Elasticsearch ≈ 900 Mo heap réduit, Kibana ≈ 450 Mo, Prometheus ≈ 600 – 900 Mo, ArgoCD ≈ 400 – 700 Mo) : total observé 4,3–5 Go sur 12 Go de cluster, marge fine documentée inline dans les manifests.

AlertManager v0.27.0 (réplique unique) route les alertes issues des PrometheusRule :

Tableau 7.1. Catalogue des alertes plateforme

Alerte	Condition	for	Gravité
SLOAvailabilityBreach	dispo. 30 j $< 99,9\%$	10 min	critical
SLOLatencyP95Breach	p95 > 200 ms	5 min	warning
SLO5xxErrorRateBreach	taux 5xx $> 1\%$	2 min	critical

Alerte	Condition	for	Gravité
ContainerMemoryRatioHigh	RAM > 80 % limite	2 min	warning
PodEvictionOngoing	OOMKill détecté	1 min	critical
HPAPinnedAtMaxReplicas	HPA au max 15 min	15 min	warning

Justification du choix technique

Des SLO chiffrés transforment la supervision subjective en contrat mesurable. Les alertes « incident » correspondent à des scénarios réellement rencontrés. Le groupement AlertManager (groupBy alertname/service, groupWait 30s) évite la tempête de notifications lors d'un incident commun à plusieurs pods.

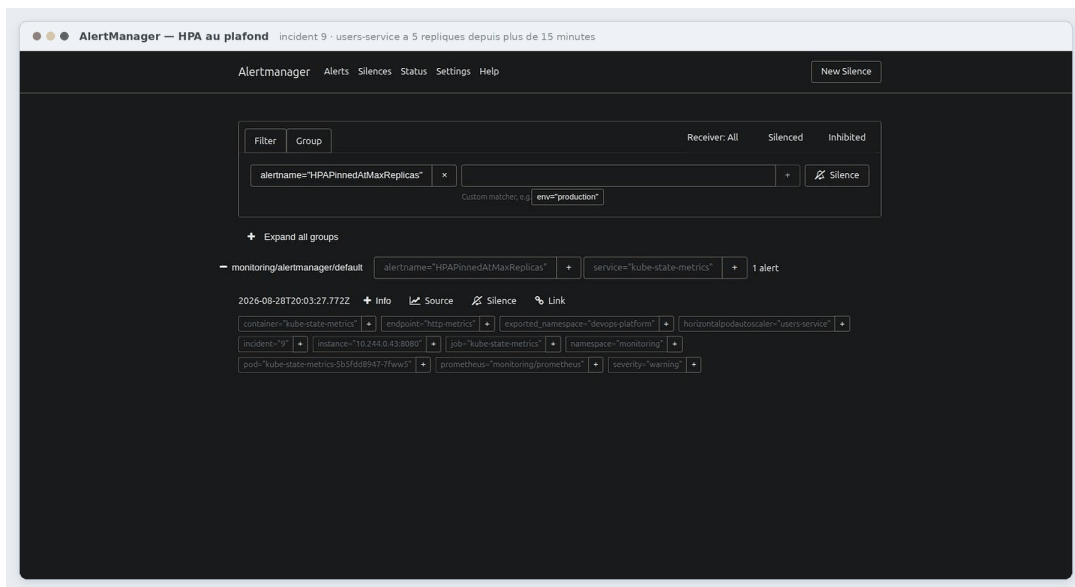


Figure 7.2. Alerte déclenchée en conditions réelles : stress-hpa.sh lancé puis HPAPinnedAtMaxReplicas firing dans AlertManager.

7.3 Grafana — visualisation dashboards-as-code

Grafana 11.1.0 : 1 réplica, admin credentials hors bande, inscription anonyme désactivée. Sidecar `k8s-sidecar` watch en continu les ConfigMaps labellisés `grafana_dashboard` : dashboards-as-code, reviewables en PR, versionnés, synchronisés par ArgoCD comme tout manifest. Deux datasources provisionnées : Prometheus (défaut) et Elasticsearch-Logs.

Tableau 7.3. Les quatre dashboards provisionnés

Dashboard	Contenu
Infrastructure Overview	Nœuds Ready, total Pods, CPU/mémoire cluster %, par namespace : refresh 30 s
Application Performance	RPS par handler, p95/p99 par endpoint, heatmap durée : refresh 15 s
Error Rate SLO	% 5xx avec seuil 1 %, compteur 5xx, % 4xx, top 5 endpoints en erreur
Infrastructure Detail	Ratio mémoire par conteneur (seuils 80/90), OOMKills, réplicas, HPA-at-max

Justification du choix technique

Le sidecar élimine l'export/import manuel JSON : la vérité reste dans Git. La double datasource métriques+logs permet de passer d'un graphe aux logs correspondants sans changer d'outil.

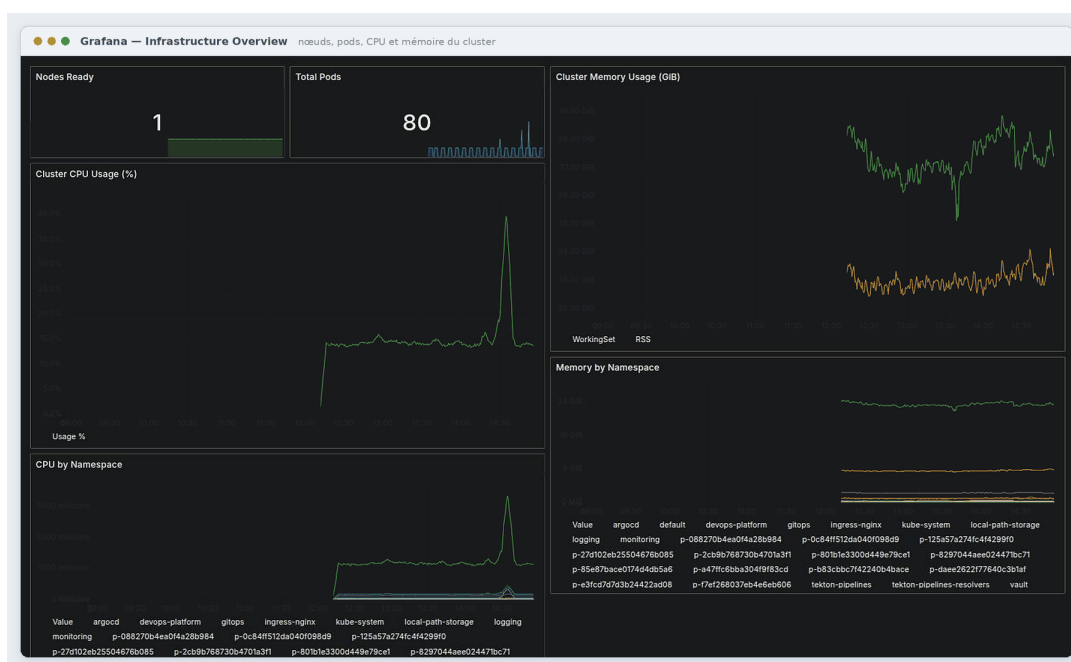


Figure 7.3. Dashboard Infrastructure Overview complet : tuiles nœuds/pods/CPU/mémoire.

7.4 ELK Stack 8.14 — logs centralisés

Les métriques disent *qu'il y a* un problème ; les logs disent *pourquoi*. Quatre composants Elasticsearch 8.14.0 coordonnés : **Filebeat** (DaemonSet, autodiscovery Kubernetes par label `part-of: devops-platform` : tout nouveau Pod de la plateforme est loggé sans configuration) → **Elasticsearch** (StatefulSet mono-nœud, heap JVM réduit 512Mo

pour tenir dans la VM 4 Go, PVC 10 Gi) → **Kibana** (exploration, NODE_OPTIONS contraint). **Logstash** est déployé en parallèle (pipeline d'enrichissement JSON + drop des healthchecks /livez//readyz), prêt à s'intercaler entre Filebeat et Elasticsearch.

Credentials créés par un script dédié qui refuse les placeholders et rend le YAML dans un fichier temporaire 0600 avec nettoyage : jamais de secret en argv (`-from-literal`).

Justification du choix technique

ELK reste la référence open-source complète (collecte, enrichissement, plein-texte, exploration). Filebeat en DaemonSet est le pattern standard de collecte nodale ; toute la stack est dimensionnée au plus juste (heaps JVM réduits, queues bornées) pour tenir dans l'homelab.

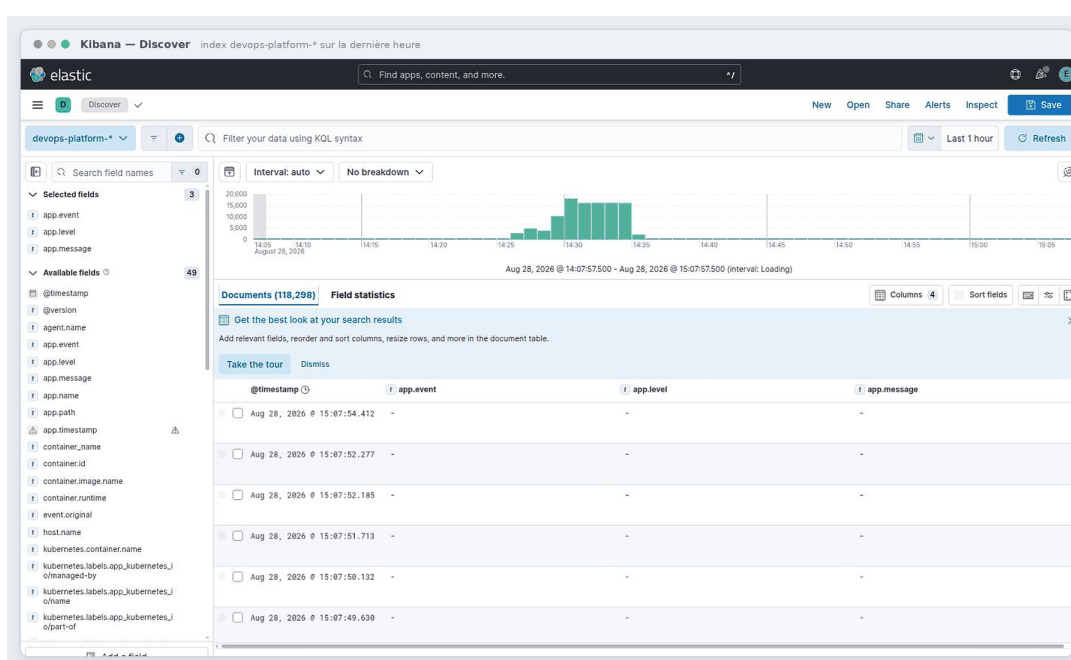


Figure 7.4. Kibana Discover : index devops-platform-* sélectionné, logs JSON du service users visibles.

7.5 Modèle de menaces simplifié et contre-mesures

Lecture croisée des outils de la plateforme sous l'angle des menaces :

Tableau 7.5. Modèle de menaces simplifié et contre-mesures

Menace	Vecteur	Contre-mesure(s) en place
Secret commité	push accidentel (clé API, .env)	pre-commit gitleaks + job CI bloquant historique complet ; allowlist resserrée
Image vulnérable	CVE OS/lib dans l'image déployée	Trivy bloquant CRITICAL/HIGH unfixed + SBOM archivé + rescan nocturne
Dépendance vulnérable	requirements.txt compromis	pip-audit -strict ×4 fichiers, CI + nocturne
Secrets exposés au cluster	Secrets K8s lisibles / tokens longs	Vault KV-v2 TTL 1h, politiques read-only par service, NetworkPolicy 8200 seule sortie
Token root divulgué	argv, disque, logs	bootstrap via stdin, mktemp 0600+trap, no_log Ansible
Conteneur privilégié	mauvais manifest	PSA restricted + confest deny ×3 + Kyverno audit + securityContext strict
Pod root filesystem modifiable	exploitation runtime	readOnlyRootFilesystem + emptyDir /tmp
Mouvement latéral	Pod compromis scanne le cluster	default-deny ingress/egress + whitelist explicite
Dérive manuelle du cluster	kubectl edit hors process	ArgoCD selfHeal + prune
État Terraform corrompu	applies concurrents	backend local isolé ; contrat S3+DynamoDB prêt
Nœud attaqué via SSH	brute-force / root login	cloud-init : root off, passwords off, fail2ban, sudo whitelist
Régistre spoofé	image substituée	GHCR permissions, déploiement épinglé par digest sha256

Justification du choix technique

Aucun contrôle ne vaut seul : c'est la superposition (IaC durcie → images scannées → secrets dynamiques → admission restreinte → réseau fermé → GitOps auto-correctif) qui forme la défense en profondeur. Le tableau ci-dessus sert aussi de checklist d'audit : chaque ligne doit rester vraie.

7.6 SLO — objectifs de niveau de service

7.6.1 Le contrat chiffré

Tableau 7.7. SLO — Le contrat chiffré

SLI	SLO
Disponibilité	$\geq 99,9\%$ sur 30 jours
Latence p95	< 200 ms
Taux d'erreurs 5xx	$< 1\%$
MTTD (délai de détection)	< 2 min
MTTR (délai de rétablissement)	< 30 min

7.6.2 Implémentation bout en bout

Ces chiffres ne sont pas un slide : ils existent à trois endroits cohérents —

1. dans les règles Prometheus (§AlertManager) : seuils exacts 0,1 % d'indispo, 0,2 s p95, 1 % 5xx ;
2. dans Grafana : ligne de seuil sur le dashboard Error Rate ;
3. dans ce rapport et la documentation plateforme.

Les MTTD/MTTR découlent des intervalles : scrape 15 s + for 2–10 min $\Rightarrow$ détection en moins de deux minutes ; auto-réparation K8s + selfHeal ArgoCD $\Rightarrow$ rétablissement typique en secondes.

7.6.3 Éléments de démonstration

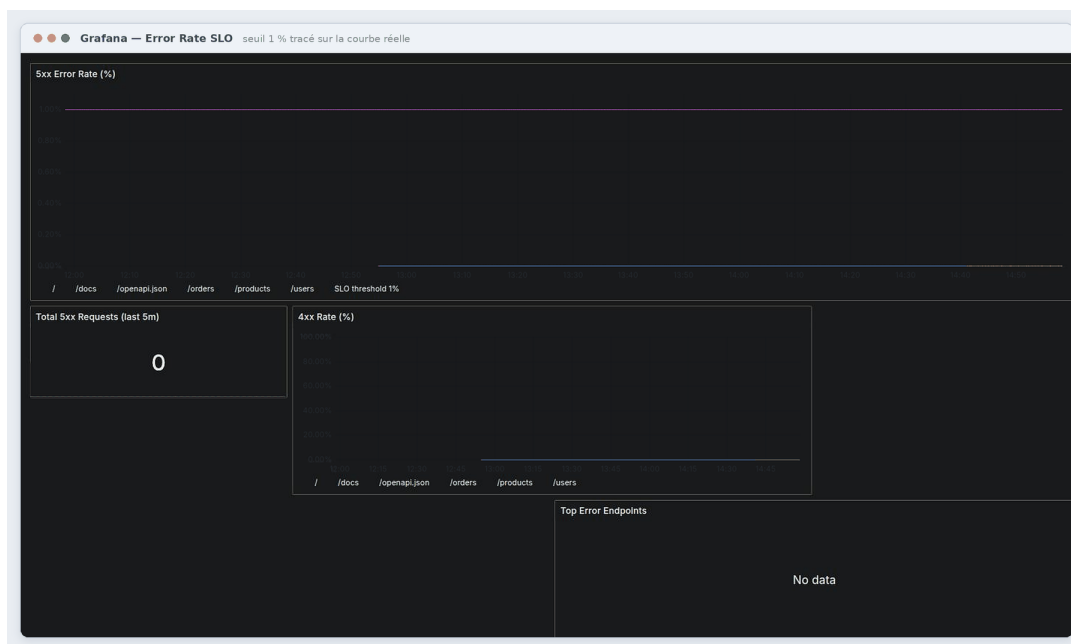


Figure 7.5. Vue Grafana Error Rate SLO annotée : seuil 1 % tracé sur la courbe réelle.

CHAPITRE 8

Scénarios de validation de bout en bout

Sept scénarios de bout en bout ont été rejoués pour valider le comportement réel de la plateforme, en condition nominale comme dégradée. Chacun part d'une action déclenchée manuellement (push, coupure d'un composant, charge) et suit la propagation jusqu'à l'observabilité (métriques, alertes, logs).

Tableau 8.1. Synthèse des scénarios de validation de bout en bout

Scénario	Déclencheur	Résultat observé
1. Commit → prod	git push sur secondary	Chaîne CI complète (lint→gitleaks→test→build→trivy) verte, ArgoCD passe OutOfSync puis Synced, rollout progressif confirmé par le nouveau digest et /metrics.
2. Panne Vault	kubectl scale deploy vault -replicas=0	Readiness bascule à 503, Service perd ses endpoints (fail-closed), livez reste OK (pas de crash), rétablissement propre au retour de Vault.
3. Fuite mémoire	Charge applicative soutenue	Ratio mémoire/limite >80 %, alerte déclenchée, OOMKill observé, post-mortem rédigé avec étiquette incident .
4. Stress HPA	Génération de charge (ab)	Scale-up 2→5 répliques, performance stable sous charge, scale-down après accalmie sans flapping (anti-flap validé).
5. Rollback GitOps	Régression volontaire déployée	Hausse du taux d'erreur détectée par alerte, revert Git, ArgoCD resynchronise automatiquement, métriques reviennent à la normale.

Scénario	Déclencheur	Résultat observé
6. Rotation secrets	Rotation complète des tokens Vault	Token one-shot généré, rolling restart des pods, preuve de continuité de service pendant la rotation.
7. Drift supply-chain	Scan nocturne programmé	Détection d'une image obsolète/CVE, run nocturne en échec (rouge), PR automatique de bump proposée.

```

Scénario 1 — Service en production endpoints et métriques disponibles 28/08/2026

devops@master-01:~/rapports$ kubectl exec -n devops-platform deploy/users-service -c users-service -- python -c "import
urllib.request as u;print('GET /users  :', u.urlopen('http://localhost:8000/users').status);print('GET /readyz  :',
u.urlopen('http://localhost:8000/readyz').status);m=u.urlopen('http://localhost:8000/metrics').read().decode();print('metri
ques  :', len([l for l in m.splitlines() if l.startswith('http_requests_total')]), 'series http_requests_total')"
GET /users  : 200
GET /readyz : 200
métriques  : 1 series http_requests_total

```

Figure 8.1. Scénario 1 : pod final au nouveau digest et `/metrics` répondant, preuve de bout en bout du pipeline commit → production.

CHAPITRE 9

Exploitation, résultats obtenus et bilan du projet

9.1 Scripts de validation de la plateforme

`validate-platform.sh` encode la définition exécutable de « ça marche » (flag `-ci` : outil manquant = FAIL, pas SKIP) :

Tableau 9.1. `validate-platform.sh` — les 7 contrôles

n°	Contrôle	Méthode
1	Cluster Kubernetes opérationnel	<code>kubectl get nodes -o json + jq</code> : tous Ready
2	Pods en Running	<code>readyReplicas == spec.replicas</code> par service
3	Aucune CVE critique	<code>trivy image --severity CRITICAL,HIGH --exit-code 1</code>
4	Aucun secret détecté	<code>gitleaks detect --source . --config .gitleaks.toml --redact</code>
5	ArgoCD synchronisé	toutes Applications Synced + Healthy
6	Dashboards Grafana actifs	<code>deploy prêt + ≥3 ConfigMaps grafana_dashboard=1</code>
7	Logs indexés ELK	<code>kibana prêt + ES sts prêt + filebeat complet + health green/yellow</code>
Bonus A	Self-healing	<code>delete</code> volontaire d'un Pod, retour attendu <30 s (destructif)

`validate-security.sh` ajoute 4 contrôles sécurité : Gitleaks (historique complet), Trivy CRITICAL/HIGH, Vault initialisé et non scellé, preuve de vie du token root (rejet des placeholders). Outils d'exploitation complémentaires : `smoke-test.sh` (parcours E2E), `stress-hpa.sh` (charge Apache Bench, scale attendu), `bootstrap-vault-secret.sh`/`bootstrap-elasticsearch-secret.sh` (secrets via stdin, jamais argv).

Justification du choix technique

Une plateforme non vérifiable n'existe pas : ces scripts servent simultanément de recette locale, de garde CI et de démonstration. Le bonus self-healing teste le comportement : pas la config.

```

Validation de la plateforme sept contrôles et bonus auto-réparation 28/08/2026

devops@master-01:~/rapport$ ./scripts/validate-platform.sh 2>&1 | tail -45
DevOps Central Platform – Phase 7 validation
mode: local
namespace: devops-platform

1. Cluster Kubernetes opérationnel
✅ PASS – Cluster opérationnel : 1/1 nœuds Ready

2. Pods en statut Running
✅ PASS – users-service : 2/2 Pods Running
✅ PASS – products-service : 2/2 Pods Running
✅ PASS – orders-service : 2/2 Pods Running

3. Aucune vulnérabilité critique (Trivy)
✅ PASS – Trivy users-service : 0 CRITICAL/HIGH
✅ PASS – Trivy products-service : 0 CRITICAL/HIGH
✅ PASS – Trivy orders-service : 0 CRITICAL/HIGH

4. Aucun secret détecté (Gitleaks)
✅ PASS – Gitleaks : aucun secret détecté

5. ArgoCD synchronisé
✅ PASS – ArgoCD : toutes les Applications Synced + Healthy

6. Dashboards Grafana actifs
✅ PASS – Grafana ready (1/1) – 4 dashboards provisionnés

7. Logs indexés dans Kibana (ELK)
✅ PASS – Kibana ready + Elasticsearch ready + Filebeat 1/1 – logs ingérés

Bonus A : Self-healing – delete Pod, expect re-creation < 30s
✅ PASS – Pod recréé en 23s (cap < 30s)

Résumé : 12 passés / 0 échoués / 0 sautés

✅ PASS – Cluster Kubernetes opérationnel
✅ PASS – Pods en statut Running
✅ PASS – Aucune vulnérabilité critique (Trivy)
✅ PASS – Aucun secret détecté (Gitleaks)
✅ PASS – ArgoCD synchronisé
✅ PASS – Dashboards Grafana actifs
✅ PASS – Logs indexés dans Kibana

7/7 tests passés – Projet VALIDÉ

```

Figure 9.1. scripts/validate-platform.sh complet : les 7 contrôles + bonus en vert avec durées.

9.2 Tests de charge et index des runbooks

stress-hpa.sh génère une charge Apache Bench (-c 80 -n 4000) contre users-service : l'HPA doit monter de 2 à 5 rélicas (CPU cible 70 % dépassée). Un job k6 nocturne complémentaire (tests/k6/load-test.js) est prévu dans le pipeline mais le fichier n'existe pas encore : première action corrective identifiée.

Tableau 9.3. Index des runbooks

Symptôme	Premiers réflexes	Outils
Pod CrashLoopBackOff	logs + describe (exit code), vérifier secrets Vault	kubectl, Kibana
Vault scellé/indispo	vault status ; readiness 503 assumé ; ne pas forcer les pods	vault CLI, AM alertes
OOMKill récurrent	ratio mémoire Grafana, remonter limites via overlay	KSM, rules.yaml
HPA bloqué au max	chercher fuite mémoire ; incident 9 post-mortem	stress-panel.py
Dérive cluster vs Git	UI ArgoCD diff ; ne jamais fixer à la main	ArgoCD selfHeal
5xx en hausse	dashboard Error Rate → handler fautif → logs filtrés	Grafana+Kibana
Image non tirée	digest présent GHCR ? netpol egress DNS ?	crictrl, netpol
Cluster injoignable	virsh list ; systemd kubelet/containerd sur nœud	libvirt, journalctl
Secret expiré (401 Vault)	rotation bootstrap-vault-secret.sh puis rollout	script + ArgoCD sync
Disque ES plein	retention/ILM, taille PVC 10Gi, purger indices vieux	__cat/indices

9.3 Tableau récapitulatif général

Tableau 9.5. Toutes les technologies du projet : version, rôle, raison du choix

Outil	Version	Ce qu'il fait ici	Pourquoi lui
Libvirt/KVM	QEMU-KVM	3 VMs q35 virtio, réseau NAT dédié	VMs réalistes gratuites, reproductibles
cloud-init	Ubuntu 22.04 img	Provisionnement + durcissement boot	Standard first-boot, durcit avant tout
Terraform	~1.5 (CI 1.5.7)	IaC réseau, volumes, VMs, inventaire	Standard IaC, plan/état/dérive
Provider libvirt	0.9.8	Piloter KVM depuis Terraform	Seul provider KVM mature
hashicorp/local	2.9.0	Rendre le fichier inventaire	Artefacts locaux idempotents
Ansible	collection 8.0.0	Config Docker, K8s, jointure, CNI	Agentless, idempotent, lisible
Docker CE	repo officiel	Builds multi-stage des images	Référence construction OCI
containerd	repo officiel	Runtime CRI des Pods	Recommandé kubeadm, léger
Kubernetes	1.28 (held)	Orchestration, autoscaling, auto-rép	API universelle orchestration
kubeadm	1.28	Cycle de vie cluster	Officiel, conforme upstream

Outil	Version	Ce qu'il fait ici	Pourquoi lui
Calico	v3.26.1 Tigera	CNI routage + NetworkPolicies	Mature, ingress+egress complets
Kustomize	5.4.3	Base + overlays env	Patch sans template, natif GitOps
FastAPI	0.139.0	Framework HTTP services	Pydantic, OpenAPI, async
Uvicorn	≥0.24 std	Serveur ASGI port 8000	Référence ASGI
hvac	2.1.0	Client Vault KV-v2	Client Python de référence
python-json-logger	2.0.7	Logs JSON stdout	Structuré ELK, minimal
prometheus-fastapi-instrumentator	6.1.0	Métriques RED /metrics	Une ligne = histogramme normalisé
Vault	1.15.2 dev mode	Secrets dynamiques, auth K8s TTL 1h	Rotation/révocation centralisée
GitHub Actions	—	Pipeline CI/CD + jobs nocturnes	CI native dépôt, matrice, OIDC
GHCR	ghcr.io	Registre images	Colocalisé, permissions héritées
Gitleaks	8.18.0	Détection secrets local+CI	Scan historique rapide, config maison
Trivy	action v0.24.0	Scan CVE bloquant, SARIF, SBOM	Scanner unifié CNCF
pip-audit	-strict	Audit requirements Python	Officiel PyPA
Ruff	0.1.9	Linter Python	Ultra-rapide, remplace 3 outils
yamllint	1.33.0	Hygiène YAML	Simple, pre-commit+CI
kubeconform	0.6.4	Validation schémas K8s 1.28	Hors ligne, rapide
conftest (Rego)	mode audit	Politiques sécurité manifests	Policy-as-code pré-admission
Kyverno	Audit	Policies admission cluster	Natif K8s, audit → enforce
pre-commit	11 hooks	Qualité avant push (5 dépôts)	Shift-left, parité CI
ArgoCD	v3.5.1	GitOps pull, prune, selfHeal, waves	État cluster = Git, rollback = revert
Prometheus Operator	v0.74.0	CRDs ServiceMonitor/PrometheusRule	Découverte déclarative GitOps
Prometheus	v2.51.0	Collecte 15 s, rétention 15 j	Standard métriques, PromQL
kube-state-metrics	v2.10.1	État objets en métriques	Source alertes HPA/OOM
AlertManager	v0.27.0	Groupement/routage alertes	Routage standard écosystème
Grafana	11.1.0	Dashboards provisionnés sidecar	Dashboards-as-code versionnés
kiwigrd sidecar	1.27.6	Chargement dashboards ConfigMap	Provisionnement auto
Elasticsearch	8.14.0	Stockage/recherche logs	Plein-texte, référence ELK
Filebeat	8.14.0	Collecte logs nœuds/Pods	Autodiscovery K8s standard
Logstash	8.14.0	Enrichissement, drop healthchecks	Transformation events
Kibana	8.14.0	Exploration logs	UI recherche standard ELK
Apache Bench	ab -c80 -n4000	Génération charge HPA	Zéro dépendance
k6	action v0.3.1	Test charge scripté nocturne	Scénarios JS versionnés (à écrire)
jq/curl	—	Manipulation JSON scripts	Bases scripting exploitation

9.4 Limites connues et axes d'amélioration

Lecture honnête du code actuel :

- `tests/k6/load-test.js` n'existe pas, le job nocturne échoue systématiquement : priorité n°1 ;
- `job deploy lit needs.build.outputs.image` (une chaîne pour 3 lignes de matrice, dernière gagnante) : limitation documentée inline ;
- `trivy-scan` résout les images par tag `github.ref_name` et non par digest malgré l'intention affichée ;
- Vault reste en dev mode (in-memory, auto-unseal, token root partagé) : raft/TLS/injector préparés mais non câblés côté Pods ;
- `overlay dev` patche réplicas à 1 sans retirer les HPA (min 2) : conflit latent ;
- AppProject ArgoCD conserve destinations `istio-system/flagger-system` héritées de la stack canary supprimée ;
- Filebeat commenté comme envoyant vers Logstash:5044 alors qu'il écrit directement Elasticsearch ;
- corruption cosmétique d'accents dans quelques chaînes de `validate-platform.sh` (logique intacte) ;
- checksum manifest Tigera non vérifié (TODO inline) ; `host_key_checking=False` assumé lab.

Chaque point est borné et actionnable : aucun ne compromet l'architecture d'ensemble ; plusieurs sont déjà documentés inline dans le code concerné.

Conclusion générale et perspectives

Bilan du travail réalisé

DevOps Central Platform démontre qu'une chaîne DevOps professionnelle complète peut tenir dans un homelab : Terraform provisionne, cloud-init durcit, Ansible installe, Docker empaquette, Kubernetes orchestre, GitHub Actions vérifie et pousse, Trivy/Gitleaks/pip-audit filtrent la supply chain, Vault distribue les secrets en fail-closed, ArgoCD maintient Git comme unique source de vérité, Prometheus/Grafana/ELK rendent le tout observable et les SLO le rendent contractuel.

Trois enseignements traversent ce rapport : **chaque outil a été choisi pour une raison précise**, opposable aux alternatives ; **la sécurité est transversale, pas additive** (boot, manifests, CI, applicatif, admission) ; **l'honnêteté technique fait partie du design** : limites listées, incidents avec leurs alertes, TODO inline.

Apport personnel

Au-delà des livrables techniques, ce projet a constitué une mise en situation d'ingénierie complète : arbitrer entre solutions concurrentes, assumer ces arbitrages par écrit, puis les confronter à l'exécution réelle : qui a régulièrement invalidé les hypothèses initiales (dimensionnement mémoire, temporisation des add-ons, stratégie de secrets). Ce travail a consolidé des compétences transversales : lecture de documentation de référence, diagnostic méthodique en situation d'incident, rédaction d'une documentation d'exploitation destinée à un tiers.

Limites du travail

HashiCorp Vault fonctionne en mode *dev*, non persistant : convient à une démonstration mais pas à une configuration de production. Les politiques Kyverno restent en mode *Audit*. Le test de charge k6 est prévu par le pipeline mais son script n'est pas encore écrit. La plateforme s'exécute sur un homelab à ressources contraintes : les chiffres de performance ne sont pas transposables tels quels à une infrastructure de production.

Perspectives d'évolution

Quatre axes prolongent naturellement ce travail : **sécurisation des secrets** (Vault en mode production, stockage persistant, auth Kubernetes par ServiceAccount, rotation automatisée) ; **politiques en mode bloquant** (Kyverno en **Enforce** après observation) ; **consolidation CI** (test de charge k6, digest plutôt que tag pour Trivy, OIDC pour le backend Terraform) ; **enrichissement applicatif** (PostgreSQL réel, vérification JWT effective). À plus long terme : extension multi-cluster, déploiement progressif (canary/blue-green) via ArgoCD Rollouts, et traçage distribué en complément des métriques et logs.

Bibliographie

- [1] G. Kim, J. Humble, P. Debois, J. Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*, IT Revolution Press, 2016.
- [2] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [3] B. Beyer, C. Jones, J. Petoff, N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016.
- [4] B. Burns, J. Beda, K. Hightower, *Kubernetes: Up and Running*, 2^e édition, O'Reilly Media, 2019.
- [5] Y. Brikman, *Terraform: Up and Running: Writing Infrastructure as Code*, 3^e édition, O'Reilly Media, 2022.
- [6] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2^e édition, O'Reilly Media, 2020.
- [7] J. Turnbull, *Monitoring with Prometheus*, Turnbull Press, 2018.
- [8] The Kubernetes Authors, *Kubernetes Documentation*, <https://kubernetes.io/docs/>, consulté en 2026.
- [9] The Kubernetes Authors, *Creating a cluster with kubeadm*, <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/>, consulté en 2026.
- [10] The Kubernetes Authors, *Pod Security Admission*, <https://kubernetes.io/docs/concepts/security/pod-security-admission/>, consulté en 2026.
- [11] HashiCorp, *Terraform Documentation*, <https://developer.hashicorp.com/terraform/docs>, consulté en 2026.
- [12] HashiCorp, *Vault Documentation*, <https://developer.hashicorp.com/vault/docs>, consulté en 2026.
- [13] Red Hat, *Ansible Documentation*, <https://docs.ansible.com/>, consulté en 2026.
- [14] Docker Inc., *Dockerfile best practices*, <https://docs.docker.com/build/building/best-practices/>, consulté en 2026.

- [15] Tigera, *Calico Documentation*, <https://docs.tigera.io/calico/latest/about/>, consulté en 2026.
- [16] Argo Project, *Argo CD: Declarative GitOps CD for Kubernetes*, <https://argo-cd.readthedocs.io/>, consulté en 2026.
- [17] OpenGitOps Working Group (CNCF), *GitOps Principles v1.0*, <https://opengitops.dev/>, consulté en 2026.
- [18] Prometheus Authors, *Prometheus Documentation*, <https://prometheus.io/docs/>, consulté en 2026.
- [19] Elastic, *Elastic Stack Documentation*, <https://www.elastic.co/guide/>, consulté en 2026.
- [20] S. Ramírez, *FastAPI Documentation*, <https://fastapi.tiangolo.com/>, consulté en 2026.
- [21] Aqua Security, *Trivy Documentation*, <https://aquasecurity.github.io/trivy/>, consulté en 2026.
- [22] OWASP Foundation, *OWASP Top 10 (2021)*, <https://owasp.org/Top10/>, consulté en 2026.
- [23] NIST, *Application Container Security Guide*, Special Publication 800-190, 2017.
- [24] Center for Internet Security, *CIS Kubernetes Benchmark*, <https://www.cisecurity.org/benchmark/kubernetes>, consulté en 2026.